Date

Page No.

Expt. No. 1

Aim:- To write a LEX program to count the numbers of vowels and consonants present in a given C file.

Theory:- Lexical analysis is the first phase of a compiler, where the input program is converted into tokens. A tool called LEX is used to generate lexical analyzers. It uses regular expressions to define patterns and corresponding actions.
In this experiments, LEX is used to identify vowels and consonants from an input file.
LEX reads the file character by character and matches patterns.
When a vowels is detected, a counter is incremented. Similarly, consonants are counted using another pattern. Non-alphabets characters are ignored.

Algorithm :-

1. Start the program.
2. Initialize vowels and consonants counters
3. Define patterns for vowels and consonants.
4. Read input file using LEX.
5. If vowels found → increment vowels count
6. If consonants found → increament consonant count
7. Ignore other characters
8. Print the final counts
9. Stop

Teacher's Signature

Oct 18, 2026, 20:46

Date

Page No.

Expt. No.

Program (LEX) :-

```
%{
#include <stdio.h>
  int vowels = 0, consonants = 0;
  %}
%%
[aeiouAEIOU]
{
  vowels++; }
[a-zA-Z];
{
  consonants++; }
   {
      }
%%
int main()
{ yylex();
  printf("Vowels : %d \n", vowels);
  printf("consonants : %d \n", consonants);
     return 0; }
```

Output :-

Inputs : Hello GIT

Vowels : 3

Consonants : 5

Teacher's Signature

Oct 18, 2026, 20:46

Date

Page No.

Expt. No.

Viva Questions :-

1. What is LEX and what is its role in compiler design?

⇒ LEX is a tool used to generate a lexical analyzer (scanner). It reads the source code, identifies patterns using regular expressions, and converts them into tokens such as keywords, identifiers, numbers and operators.

2. How do you differentiate blw vowels and consonants in your LEX program?

⇒ We use different regular expression patterns for vowels and consonants.

for eg.

```
[aeiou AEIOU]   {printf ("Vowel");}
[a-z A-Z]       {printf ("consonant");}
```

The first pattern matches vowels. other alphabetic characters are treated as consonants.

3. How does LEX handle overlapping patterns?

⇒ LEX follows two main rules :-

1. It selects the pattern that matches the longest input string.
2. If two patterns match the same length, it selects the rule that appears first in LEX program.

This is called the longest-match rule.

Teacher's Signature

Oct 18, 2026, 20:46

Expt. No.

Date

Page No.

4. What is the purpose of the yylex() function?

⇒ yylex() is the main function generated by LEX. It scans the input, matches it with the defined patterns, and performs the corresponding actions.

5. How do you handle uppercase and lowercase letters in your pattern matching?

⇒ We include both uppercase and lowercase letters in regular expressions.

for example :- [aeiouAEIOU]

Here, aeiou matches lowercase vowels and AEIOU matches uppercase vowels.

Teacher's Signature

31.7.25

Oct 18, 2026, 20:46

Expt. No. 2

Date

Page No.

Aim:- Write a LEX program to count the number of blank space, words and lines in a ~~tifo~~ file.

Algorithm:-

1. Start
2. Initialize counters for words, space and line.
3. Define pattern for words, space and new line.
4. Read file using LEX
5. Increment counters accordingly
6. Print results
7. Stop

Program (LEX):-

```
%{
  # include <stdio.h>
  int words = 0, spaces = 0, lines = 0;
%}
%%
[\t] + { spaces ++;
  }
  \n { line ++;
  }
[a-z A-Z] +
{
  words ++;
}
{
}
%%
```

Teacher's Signature

Oct 18, 2026, 20:46

Date

Page No.

Expt. No.

```
int main ()
{ yylex();
printf ("words : %d \n", words);
 printf ("spaces : %d \n", spaces);
 printf ("Lines : %d \n", lines);
     return 0; }
```

Viva Question :-

1. How do you define a word in your LEX program?
In LEX, a word is defined using a regular expression
A common pattern is

→ [A-Za-Z0-9]+

→ [^ \t \n]+

This matches one or more consecutive characters that are not spaces, tabs or newlines. Whenever this pattern matches, the word counter is incremented.

2. How do you Count Blank lines ?

Ans A Blank line is a line that Contain only a newline (\n) or white space followed by a newline In LEX. we can detect it using a pattern like.

^[\t]* \n

when this pattern matches, we increase the blank line Conter.

Teacher's Signature

Oct 18, 2026, 20:46

Date

Page No.

Expt. No.

3. What is the difference b/w counting words and counting lines?

Ans. Word counting :- Counts every valid word in the input using a word pattern.

Line counting :- Counts every newline character (\n) to determine the total no. of lines.

eg: Hello world
LEX Program
• words = 4
• lines = 2

4. How do you handle multiple consecutive spaces?

Ans. Multiple consecutive spaces are treated as a single separator b/w words. In LEX, spaces and tab are ignored using a rule such as.

[\t] +

The (+) operator matches one or more spaces or tabs, ensuring that extra spaces do not increase the word count.

5. What is the purpose of yywrap() function in LEX programs?

Ans. The yywrap() function is called when LEX reaches the end of the input file.

• If it returns 1, scanning stops because there is no more input.

• If it returns 0, LEX continues scanning another input file.

→ A common implementation is

→ int yywrap()
{return 1;}

This tells LEX to terminates after processing the current file.

Teacher's Signature

Oct 18, 2026, 20:46

Expt. No. 3

Date

Page No.

Oct 18, 2026, 20:46

Aim :- To write a LEX program to recognize identifiers, keywords, and operators.

Theory :- In programming languages, identifiers are names given to variables and functions. Keywords are reserved words like int, float etc. Operators are symbols used to perform operations.

Algorithm :-

1. Start
2. Define Keyword list
3. Define regex for identifiers and operators
4. Read inputs using LEX
5. Match patterns and print type
6. Stop

Program (LEX) :->

```
%{
# include <stdio.h>
%}
%%
"int" | "float" | "if" | "else"
{ printf ("Keyword : %s \n", yytext); }
[a-zA-Z][a-zA-Z0-9]*
{ printf ("Identifier : %s \n", yytext); }
[+\-*/=]
{ printf ("Operator : %s \n", yytext); }
[\t\n]+
{ }
{} %%
```

Output :-

Input : int a = b + c

Keyword : int

Identifier : a

Operator : =

Identifier : b

Operator : +

Identifier : c

Teacher's Signature

Date

Expt. No. Page No.

```
int main ()
{ yylex ();
  return 0; }
```

Viva Question :-

1. How do you differentiate between Keywords and identifiers in LEX?

⇒ Keywords are reserved words with predefined meaning. (eg - if, while, int). Identifiers are user-defined names for variables, functions etc.

2. What regular expression do you use to recognize identifiers?

Ans [a-z A-Z _ [a-z A-Z 0-9 _]*

It means:
→ first character → letter or underscore
→ Remaining character → letter, digits or underscore.

3. How do you handle coperators like ++ and == ?

Ans We define separate LEX rules for multi-character operators, for example :-

```
"++" { printf ("operator : %s \n", yytext); }
"==" { printf ("operator : %s \n", yytext); }
```

Teacher's Signature

Oct 18, 2026, 20:47

Date

Page No.

Expt. No.

4. What is the difference between a token and a lexeme ?

Ans Token :- The category / type of a lexical unit.

lexemme :- The actual sequence of characters in the source program.

5. How do you handle escape sequence and string literals ?

Ans Escape sequence such as \n, \t, \", and \" are handled by defining appropriate LEX patterns. String literals can be recognised using a pattern that starts and ends with double quotes and allows escaped character inside them.

xxx xxx xxx

Result 30/9/25

Teacher's Signature

Oct 18, 2026, 20:47

Output :-

Input : 9933164180

Valid Mobile Number

Date

Expt. No. 4

Page No.

Aim :- To write a LEX program to validate mobile number, URL, date and time.

Theory :- Validation ensures that input follows a specific format. LEX uses regular expressions to validate patterns like mobile numbers, URLs, dates, and time.

Algorithm :-

1. Start
2. Define regex patterns
3. Read input
4. Match with patterns
5. Display valid / invalid
6. Stop

Program (LEX) :-

```
%{
#include <stdio.h>
%}
%%
[0-9]{10}
{printf("Valid Mobile Number\n");}
(http|https)://[a-zA-Z0-9./];
{printf("Valid URL\n");}
[0-9]{2}/[0-9]{2}/[0-9]
{4}
{printf("Valid Date\n");}
[0-9]{2}:[0-9]{2}
{printf("Valid Time\n");}
.{printf("Invalid Input\n");}
%%
```

Teacher's Signature

Oct 18, 2026, 20:47

Date

Page No.

Expt. No.

```
int main()
{ yylex();
  return 0; }
```

Viva Questions :-

1. What regular expression do you use to validate an Indian mobile number?

⇒ A common regular expression is :- ^[6-9][0-9]{9}$
It checks that the number has 10 digits and starts with 6, 7, 8, or 9.

2. How do you validate a URL in LEX?

⇒ In LEX, a URL can be validated using a regular expression that checks for the protocol (http/https), domain name and optional path.

3. What are the different date formats you validate?

⇒ Common date formates include :-

- DD/MM/YYYY → 20/08/2026
- MM/DD/YYYY → 08/20/2026
- YYYY-MM-DD → 2026-08-20
- DD-MM-YYYY → 20-08-2026

4. How do you handle 12 hour v/s 24-hour time formats?

Teacher's Signature

Oct 18, 2026, 20:47

Expt. No. _____ Page No. _____

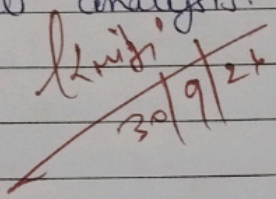
→ 12-hour format :- Uses AM or PM, e.g., 10:30 PM.

→ 24- hour format :- Uses hours from 00 to 23, e.g., 22:30.

We use different regular expressions to validate each format.

5. What is the purpose of start conditions in validating complex patterns?

⇒ Start conditions help LEX to perform context - dependent lexical analysis.



Teacher's Signature _____

Oct 18, 2026, 20:47

Expt. No. 5

Aim :- To write a C program to check whether a string is a Keyword or not.

Keywords are reserved words in C language with predefined meanings. They cannot be used as identifiers. To check whether a string is a keyword, its is compared with a list of predefined keywords. If a match is found, it is a keyword. Otherwise, it's not.

Algorithm :-

1. Start
2. Input string
3. Compare with keyword list
4. If match → keyword
5. Else → not keyword
6. Stop

Program (C)

```
# include <stdio.h>
# include <string.h>
 int main()
{
   char str[20];
   char keywords[][10] =
{"int","float","if","else","while","for","return"};
      int i, flag = 0;
      printf("Enter a string :");
      scanf("%s", str);
```

Teacher's Signature

Oct 18, 2026, 20:47

Output :-

Input : int

Keyword

Date

Page

Expt. No.

```
for (i=0; i<7; i++){
    if (strcmp (str, Keywords[i]) == 0)
{
    flag = 1;
    break; } }
if (flag)
    printf ("Keyword \n");
else
    printf ("Not a Keyword \n");
return 0; }
```

Viva Questions :-

1. What defines a Keyword in a programming language?
⇒ A Keyword is a reserved word with a predifined meaning in language.

2. How do you implement Keyword checking efficiently?
⇒ Store Keywords in an array and compare the Input using strcmp().

3. Why can't Keywords be used as identifiers?
⇒ Because they already have a special meaning in the language.

4. How does your program handle case sensitivity?
⇒ It is case-sensitive. for example, int is a Keyword, but Int is not.

5. What is the difference between a Keyword and a reserved word?
⇒ A Keyword has a specific meaning in the language, while a reserved word is set aside and cannot be used as an identifier, even if it may not currently have a specific function.

Teacher's Signature

Date

Expt. No. 6

Page No.

Aim :- To write a C program to count the total number of Keywords present in a file.

Theory :- Keywords are reserved words in C that have predefined meanings and cannot be used as identifiers. Example include int, float, if, else, while etc.
In this experiment, a file is read word by word and each word is compared with a predefined list of Keyword.

Algorithm :-

1. Start
2. Open file in read mode
3. Initialize Keyword counter
4. Read words from file
5. Compare each word with Keyword list
6. If match found → increment counter
7. Repeat until EOF
8. Print total Keywords
9. Close file
10. Stop

C program :→

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

Teacher's Signature

Oct 18, 2026, 20:47

Date

Expt. No. Page No.

```
int main () {
  FILE * fp ;
  char word [50];
    int count = 0;
char * Keywords [] = { "auto", "break", "case", "else",
                "continue", "while", "do", "double", "int" };
int n = size of (Keywords) / sizeof (Keywords [0]) ;
fp = fopen ("input.c", "r");
  if (fp == NULL) {
      printf ("Unable to open file . \n");
       return 1 ; }
while (fscanf (fp, "%49 s", word) == 1) {
  int len = strlen (word) ;
while (len > 0 && ispunct ((unsigned char) word
    [len - 1 ])) {
        word [--len] = '\0';
}
  char * start = word ;
while (* start && ispunct ((unsigned char) * start)) {
        start ++ ; }
for (int i = 0 ; i < n ; i ++) {
    if (strcmp (start, Keywords [i]) == 0) {
            count ++ ;
            break ; } } }
  fclose (fp);
printf ("number of Keywords in the file = %d \n",
                        count) ;
  return 0 ; }
```

Teacher's Signature

Oct 18, 2026, 20:47

Date

Page No.

Expt. No.

```
int main () {
    int a = 10;
    if (a > 5) {
        return 0; }}
```

Viva Questions :-

1. How does your program identify Keywords within source file?

⇒ It compares each word in the source file with a predefined list of C Keywords.

2. How do you avoid counting Keywords inside string literals?

⇒ The program detects string and ignores their contents while checking Keywords.

3. How do you handle comments?

⇒ Comments are ignored so Keywords written inside comments are not counted.

4. Difference between Keywords and reserved words?

⇒ Keywords have special meaning in C while reserved words are reserved by languages and can't be used as identifiers.

5. How does your program handle preprocessor directives? If

⇒ identifies lines beginning with # and ignores them while c
C Keywords.

Teacher's Signature

Oct 18, 2026, 20:48

1:54 4G+ 62%

EXPERIMENT – 7

Count number of operators in a file

AIM:-To write a C program to count the number of operators present in a file.

THEORY:- Operators are symbols used to perform operations on variables and values in a program. Examples include arithmetic operators (+, -, *, /), relational operators (<, >), and assignment operators (=).
In this experiment, the input file is read character by character. Each character is checked whether it belongs to the set of valid operators. If it matches, a counter is incremented.
The program uses file handling functions such as fopen(), fgetc(), and fclose(). A helper function is used to identify operators.
This experiment helps understand how tokens like operators can be identified during lexical analysis.

ALGORITHM

1. Start
2. Open file in read mode
3. Initialize operator counter
4. Read file character by character
5. Check if character is an operator
6. If yes → increment counter
7. Repeat until EOF
8. Print total operators
9. Close file
10. Stop

Program (C)

```
#include <stdio.h>
#include <string.h>

// Function to check if character is operator
int isOperator(char ch)
{
   char operators[] = "+-*/=%<>";
   for(int i = 0; i < strlen(operators); i++)
{
      if(ch == operators[i])
         return 1;
   }
   return 0;
}
```

1:54 4G+ 62%

```
{
    if(ch == operators[i])
        return 1;
  }
  return 0;
}
```

```
int main()
 {
  FILE *fp;
  char ch;
  int count = 0;
  fp = fopen("input.txt", "r");
  if(fp == NULL)
{
    printf("Error opening file!\n");
    return 1;
  }

  while((ch = fgetc(fp)) != EOF)
{
    if(isOperator(ch))
{
      count++;
      printf("Operator found: %c\n", ch);
    }
  }

  printf("\nTotal Operators = %d\n", count);

  fclose(fp);
  return 0;
}
```

Output

Input File: a = b + c * d;

Operator found: =
Operator found: +
Operator found: *

Total Operators = 3

1:54 4G+ 62%
CD La...

```
    printf("Error opening file!\n");
    return 1;
  }

  while((ch = fgetc(fp)) != EOF)
{
    if(isOperator(ch))
{
      count++;
      printf("Operator found: %c\n", ch);
    }
  }

  printf("\nTotal Operators = %d\n", count);

  fclose(fp);
  return 0;
}
```

Output

Input File: a = b + c * d;

Operator found: =
Operator found: +
Operator found: *

Total Operators = 3

Viva Questions

1. How do you define operators for counting purposes?
2. How do you differentiate between operators and other symbols?
3. How do you handle multi-character operators like ++ and ==?
4. How do you avoid counting operators inside comments and strings?
5. How does your program handle unary vs binary operators?

EXPERIMENT – 8

Count occurrence of each character in a file

AIM :-To write a C program to count the occurrence of each character in a file.

THEORY:- Character frequency analysis is the process of counting how many times each character appears in a file. It is widely used in text processing, cryptography, and data analysis.
In this program, an array of size 256 (ASCII characters) is used. Each index represents a character, and its value stores the count.
The file is read character by character using fgetc(). Each character's ASCII value is used as an index to increment its count.
This method is efficient and simple, making it suitable for analyzing text data.

ALGORITHM

1. Start
2. Open file
3. Initialize array of size 256 with zero
4. Read characters one by one
5. Increment corresponding array index
6. Repeat until EOF
7. Print frequencies
8. Close file
9. Stop

1:57 पे 4G+ 61%

9. Stop

Program (C)

```
#include <stdio.h>
int main()
{
   FILE *fp;
   int freq[256] = {0};
   char ch;

   fp = fopen("input.txt", "r");

   if(fp == NULL)
{
      printf("Error opening file!\n");
      return 1;
   }
   while((ch = fgetc(fp)) != EOF)
{
      freq[(unsigned char)ch]++;
   }
   printf("Character Frequencies:\n");

   for(int i = 0; i < 256; i++)
{
      if(freq[i] > 0)
{
         printf("%c = %d\n", i, freq[i]);
      }
   }

   fclose(fp);
   return 0;
}
```

Output

Input File: Laxmikant Vashishtha

Character Frequencies:

L = 1
a = 3
x= 1
m = 1
i = 2
k = 1
n = 1
t = 2
v = 1
s = 2
h = 3

Viva Questions

1. How do you define an identifier in your program?
2. How do you distinguish identifiers from keywords?
3. How do you avoid counting identifiers inside comments and strings?
4. How does your program handle case sensitivity in identifiers?
5. What is the significance of the underscore character in identifiers?

EXPERIMENT – 9

Symbol Table Implementation

AIM:-To implement a symbol table using C.

THEORY:- A symbol table is a data structure used by compilers to store information about identifiers such as variables, functions, and arrays.
It contains details like name, type, scope, and memory location. The symbol table is used during various phases of compilation, especially semantic analysis and code generation.
In this experiment, a simple symbol table is implemented using structures. Entries are inserted manually and displayed.
This helps understand how compilers manage identifiers efficiently.

ALGORITHM

1. Start
2. Define structure for symbol table
3. Insert entries
4. Store name and type
5. Display table
6. Stop

Program (C)

```
#include <stdio.h>
#include <string.h>

struct symbol
{
   char name[20];
   char type[10];
};

struct symbol table[100];
int count = 0;
// Insert symbol
void insert(char name[], char type[])
{
   strcpy(table[count].name, name);
   strcpy(table[count].type, type);
   count++;
}
// Display table
void display() {
   printf("\nSymbol Table:\n");
   printf("Name\tType\n");
   for(int i = 0; i < count; i++)
{
      printf("%s\t%s\n", table[i].name, table[i].type);
   }
}
```



```
int main()
{
   insert("a", "int");
   insert("b", "float");
   insert("sum", "function");

   display();
   return 0;
}
```

Output

Symbol Table:

Name	Type
a	int
b	float
sum	function

3:20 4G+ 54%

```
int count = 0;
// Insert symbol
void insert(char name[], char type[])
{
   strcpy(table[count].name, name);
   strcpy(table[count].type, type);
   count++;
}
// Display table
void display() {
   printf("\nSymbol Table:\n");
   printf("Name\tType\n");
   for(int i = 0; i < count; i++)
{
      printf("%s\t%s\n", table[i].name, table[i].type);
   }
}
```



```
int main()
{
   insert("a", "int");
   insert("b", "float");
   insert("sum", "function");

   display();
   return 0;
}
```

Output

Symbol Table:

Name	Type
a	int
b	float
sum	function

Viva Questions

1. What are the primary responsibilities of a lexical analyzer?
2. How does the lexical analyzer communicate with the parser?
3. What is the input and output of your lexical analyzer?
4. How do you implement the state transitions in your manual lexical analyzer?
5. How do you handle lexical errors like invalid characters?

EXPERIMENT – 10

YACC program to recognize strings of the form a^nb^n

AIM:- To write a YACC program to recognize strings of the form a^nb^n.

THEORY:- The language a^nb^n consists of strings where the number of a's is equal to the number of b's. Examples include ab, aabb, aaabbb.
This type of language is context-free and cannot be recognized using regular expressions alone. YACC is used to define grammar rules for such patterns.
LEX is used to generate tokens, and YACC processes them using grammar rules. The parser checks whether the input string follows the defined grammar.
This experiment demonstrates syntax analysis using YACC.

ALGORITHM

1. Start
2. Define grammar rules
3. Use LEX to generate tokens
4. Parse input using YACC
5. If grammar matches → valid
6. Else → invalid
7. Stop

Program (LEX + YACC)

LEX File

```
%{
#include "y.tab.h"
%}
%%
a   { return A; }
b   { return B; }
\n  { return 0; }
%%
int yywrap()
{ return 1; }
```

YACC File

```
%{
#include <stdio.h>
int yylex();
void yyerror(const char *s);
%}
%token A B
%%
S : A S B
  | A B
  ;
%%
```



```
int main()
{
   printf("Enter string:\n");
   yyparse();
   printf("Valid String\n");
   return 0;
}

void yyerror(const char *s)
{
   printf("Invalid String\n");
}
```

Output

Input: aabb

Valid String

```
%%
a   { return A; }
b   { return B; }
\n  { return 0; }
%%
int yywrap()
{ return 1; }
```

YACC File

```
%{
#include <stdio.h>
int yylex();
void yyerror(const char *s);
%}
%token A B
%%
S : A S B
  | A B
  ;
%%
```



```
int main()
{
   printf("Enter string:\n");
   yyparse();
   printf("Valid String\n");
   return 0;
}

void yyerror(const char *s)
{
   printf("Invalid String\n");
}
```

Output

Input: aabb

Valid String

Viva Questions

1. How does your C program differ from the LEX-based implementation?
2. What logic do you use to identify word boundaries?
3. How do you handle different types of whitespace characters?
4. What is the importance of the EOF (End-of-File) marker in your program?
5. How do you ensure your program handles empty files correctly?